23. 4 Bit Ripple Carry Counter

Test bench

```
module stimulus;
reg clk, reset;
wire [3:0] q;
ripple_counter uut (.q(q),.clk(clk),.reset(reset));

initial
        clk = 0;

always
        #5clk=~clk;

initial
begin
                reset = 1'b1;
        #15     reset = 1'b0;
        #180    reset = 1'b1;
        #10     reset = 1'b0;

        #20 $finish;
        end

initial
        $monitor($time," Output q =%d",q);
Endmodule
```

Ripple counter

```
module ripple_counter(q,clk,reset);
input clk,reset;
output [3:0]q;

t_ff A1 (q[0],clk, reset);
t_ff A2 (q[1],q[0],reset);
t_ff A3 (q[2],q[1],reset);
t_ff A4 (q[3],q[2],reset);
endmodule
```

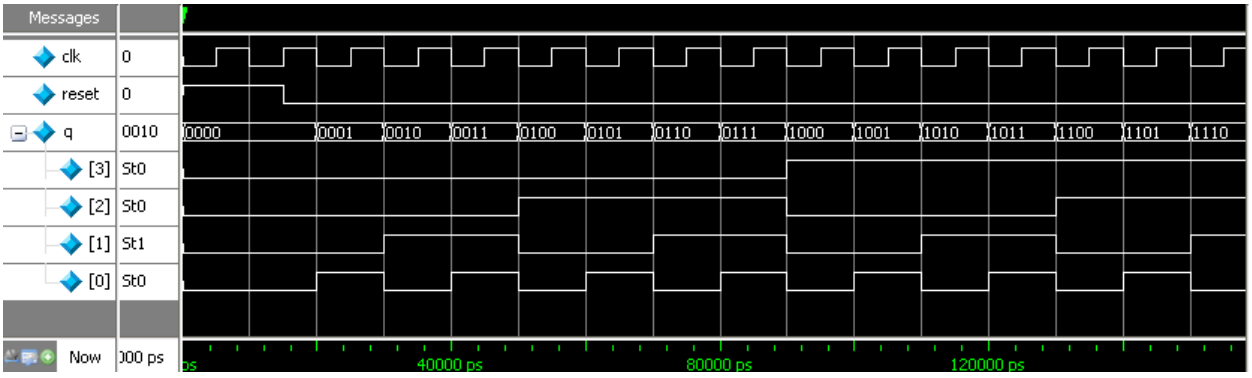
T Flip Flop

```
module t_ff (q,clk,reset);
input clk,reset;
output q;
wire d;
d_ff A1(q,d,clk,reset);
not A2 (d,q);
endmodule
```

D Flip Flop

```
module d_ff (q,d,clk,reset);
input d,clk,reset;
output reg q;

always@(posedge reset,negedge clk)
 begin
          if(reset)
                    q <=1'b0;
          else
                    q <=d;
         end
endmodule
```



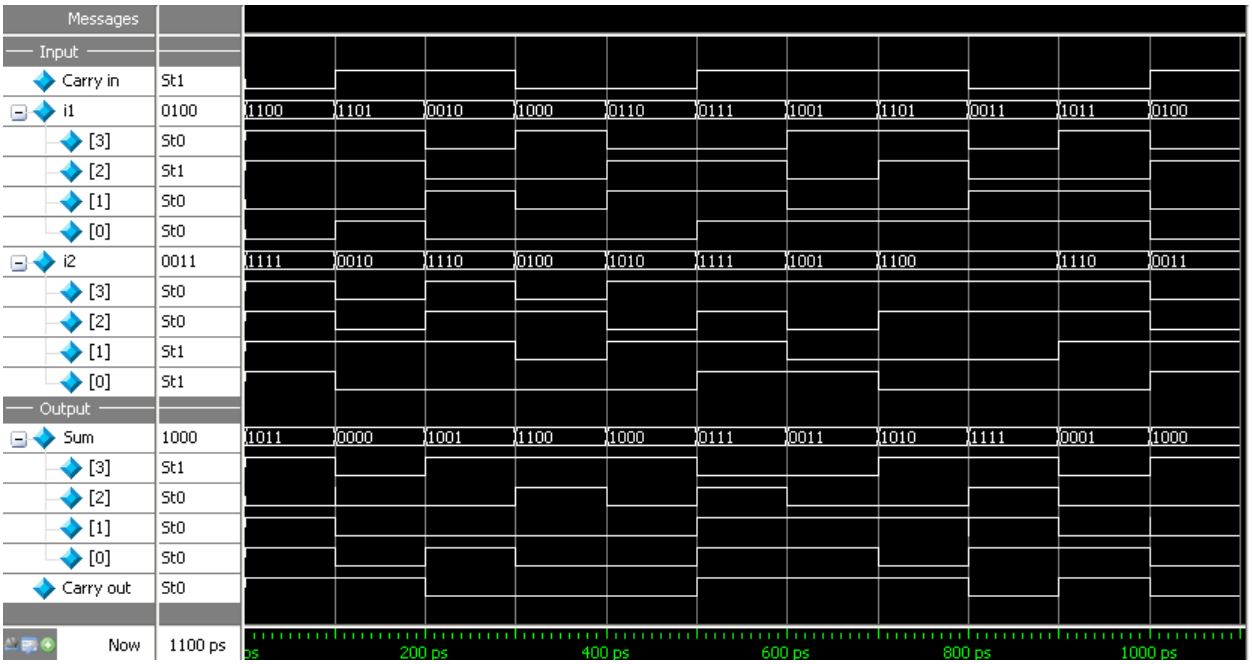
24. Carry Look Ahead Adder using dataflow modeling

```
module cla(
input [3:0] i1,i2,
input c0,
output [3:0] s,
output cout
);
wire[3:0] g,p;
wire [3:1]c;

 assign g[3:0]=i1[3:0]&i2[3:0];
 assign p[3:0]=i1[3:0]^i2[3:0];

 assign c[1]=g[0] | p[0]&c0;
 assign c[2]=g[1] | (g[1] | p[0]&c0)&p[1];
 assign c[3]=g[2] | (g[2] | (g[1] | p[0]&c0)&p[1])&p[2];
 assign cout =g[3] | (g[2] | (g[2] | (g[1] | p[0]&c0)&p[1])&p[2])&p[3];

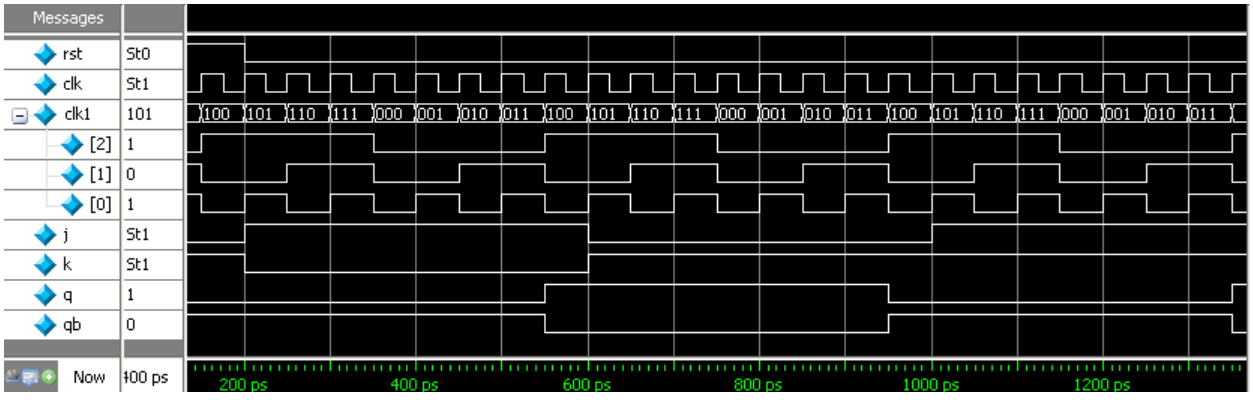
 assign s[0]=p[0]^c0;
 assign s[3:1]=p[3:1]^c[3:1];
endmodule
```



25. JK Flip Flop using behavioral modeling with clock division

```
module jk_ff(
input j,k,clk,rst,
output reg q,qb);
reg [16:0] clk1=17'd0;// reg [2:0] clk1=3'd0;
always@(posedge clk)
begin
        clk1=clk1+1;
end
always@ (q)
begin
        qb=~q;
end

initial
begin
        q<=1'b0;
end
always@(posedge clk1[16] or posedge rst)  // always@(posedge clk1[2] or
posedge rst)
begin
        if(rst)
        q<=1'b0;
else
case({j,k})
0:q<=q;
1:q<=1'b0;
2:q<=1'b1;
3:q<=qb;
endcase
end
endmodule
```



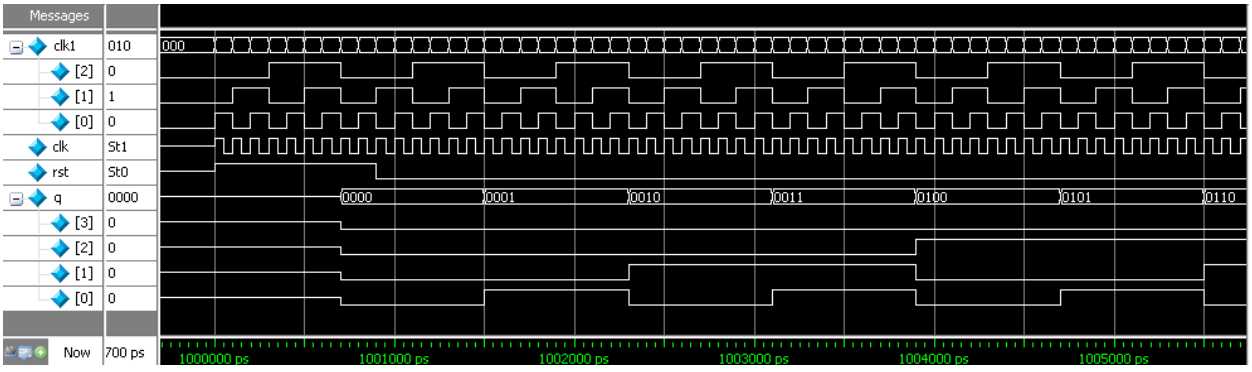
26. 4 Bit Up Counter with clock division

```
module up_counter(q, clk,rst);
output reg [3:0]q;
input clk,rst;
reg [16:0]clk1=17'd0;
 //reg [2:0]clk1=3'd0;

 always@(posedge clk)
begin
        clk1=clk1+1;
end

always@(negedge clk1[16])
//always@(negedge clk1[2])
 if(rst)
        q=0;
 else
        q=q+1;

endmodule
```



27. BCD Counter with clock division

```
module bcd_counter(
input clk,rst,
output reg [3:0] q );
reg [16:0]clk1=17'd0;  //reg [2:0]clk1=3'd0;

always@(posedge clk)
begin
        clk1=clk1+1;
end

 always@(negedge clk1[16])  // always@(negedge clk1[2])
 if( (rst==1) | (q==9) )
        q<=0;
 else
        q<=q+1;
endmodule
```

28. Any sequence Counter with clock division

```
module any_count(
output reg [3:0] q,
input clk,rst);
reg [2:0]clk1=3'd0; //reg [16:0]clk1=17'd0;

 always@(posedge clk)
begin
        clk1=clk1+1;
end

 always@(posedge clk1[2] or negedge rst)
 begin
 if(rst)
        q=4'd2;
 else
        case(q)
        2 : q=4'b0101;
        5 : q=4'b0111;
        7 : q=4'b1000;
        8 : q=4'b1001;
        9 : q=4'b0010;
        default : q=0;
        endcase
end
endmodule
```

29. Shift-Right Shift-Left Register

```
module slsr(
input sl,sr,din,clk,rst,
output reg [3:0] q);
 reg [2:0]clk1=3'd0;   //reg [16:0]clk1=17'd0;

 always@(posedge clk)
begin
        clk1=clk1+1;
end

always@(posedge clk1[2])        //always@(posedge clk1[16])
if(~rst)
begin
        if(sl)
                q<={q[2:0],din};
        else  if(sr)
                q<={din,q[3:1]};
end

else
        q=4'd0;

endmodule
```

30. Up-Down Counter with clock division

```
module up_down_counter(
input clk,updown,
output reg [3:0] q );
 reg [16:0] clk1=17'd0;
// reg [2:0] clk1=2'd0;

 always@(negedge clk)
begin
        clk1=clk1+1;
end

initial
q=4'd5;

// always@(posedge clk1[2] or updown)
 always@(negedge clk1[16])
if(updown==1)
        begin
                q=q+1;
                if(q>12)
                q=12;
        end
else
        begin
                q=q-1;
                if(q<5)
                q=5;
end
endmodule
```

31. BCD to 7 Segment Display Decoder

```
module bcd_7seg(
input [3:0] bcd,
output reg [1:7] led);
 always@(bcd)

 case(bcd)
        0 : led = 7'b1111110;
        1 : led = 7'b0110000;
        2 : led = 7'b1101101;
        3 : led = 7'b1111001;
        4 : led = 7'b0110011;
        5 : led = 7'b1011011;
        6 : led = 7'b1011111;
        7 : led = 7'b1110000;
        8 : led = 7'b1111111;
        9 : led = 7'b1111011;
        default:led = 7'bx;
 endcase

endmodule
```

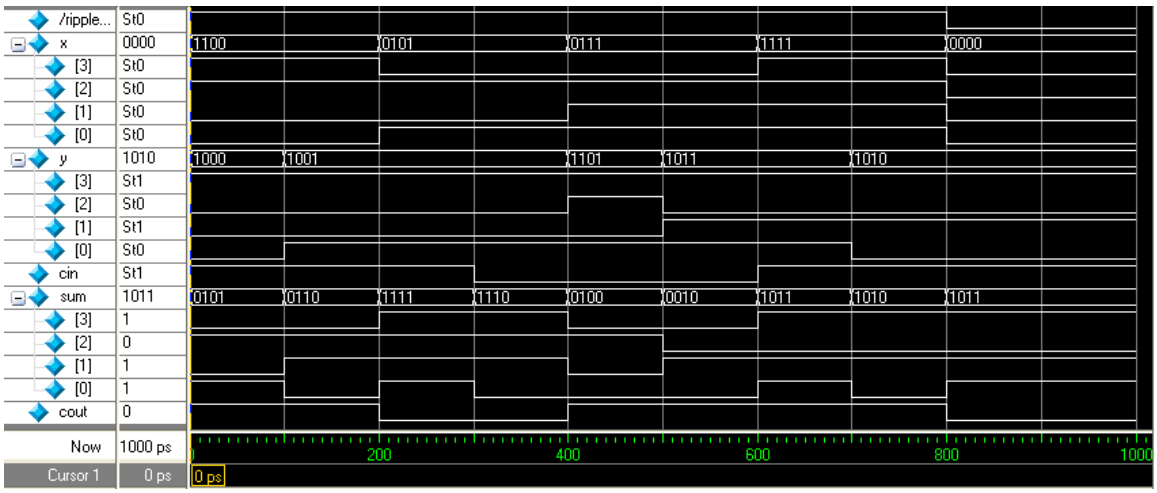
32.Binary counter using led display

```
module countdis(clk,rst, led);
   input clk,rst;
   output [1:7] led;
        reg [1:7]led;
        reg [3:0]count;
        reg [16:0] clk1=17'd0;
        always@(posedge clk)
        clk1=clk1+1;
        always@(posedge clk1[16])
        begin
        if(rst)
        count<=4'b0000;
        else
        count<=count+1;
        end
        always@(count)
        case(count)
        0:led=7'b1111110;
        1:led=7'b0110000;
        2:led=7'b1101101;
        3:led=7'b1111001;
        4:led=7'b0110011;
        5:led=7'b1011011;
        6:led=7'b1011111;
        7:led=7'b1110000;
        8:led=7'b1111111;
        9:led=7'b1111011;
        10:led=7'b1110111;
        11:led=7'b1111111;
        12:led=7'b1001110;
        13:led=7'b1111110;
        14:led=7'b1001111;
        15:led=7'b1000111;
        default:led=7'bx;
        endcase

endmodule
```

33.Ripple Carry Adder Using Task

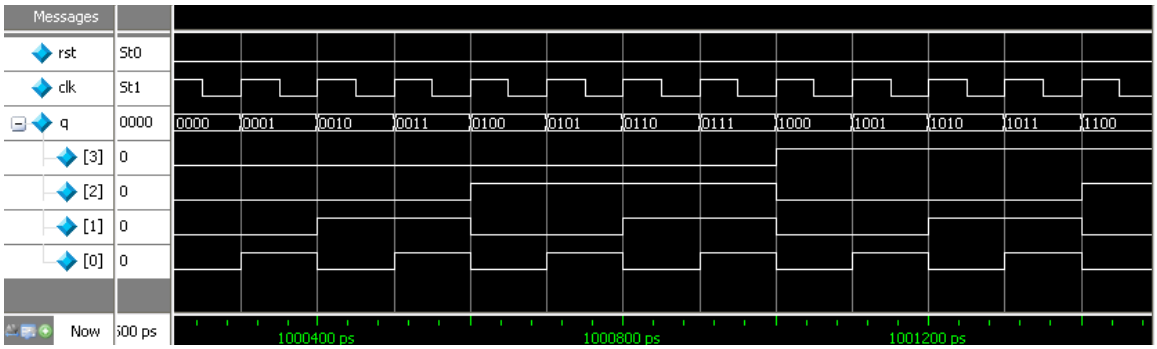
```
module rip(x,y, cin, sum, cout);
   input [2:0] x,y;
   input cin;
   output [2:0] sum;
   output cout;
         reg [2:0] sum;
   reg cout;
   reg [2:0] carry;
         integer i;
         //genvar i;
   always@(x,y,cin)
          begin
          carry[0]=cin;
          //generate
          for(i=0;i<3;i=i+1)
          begin:full_add
            Fadder(sum[i],carry[i+1],x[i],y[i],carry[i]);
                end
        //endgenerate
          cout=carry[2];
end
task Fadder;
          output sf,cf;
          input af,bf,cinf;
          begin
            sf=af^bf^cinf;
                cf=(af&bf)|(af&cinf)|(bf&cinf);
          end
         endtask
endmodule
```



34.Mod 13 Counter

```
module mod13(clk,rst, q);
   input clk,rst;
   output [3:0] q;
         reg [3:0] q;
         always@(posedge clk)
         if(rst||(q+1)%13==0)
         q<=0;
         else
         q<=q+1;

endmodule
```



35.Gray Counter

```
module gray(clk,rst,en, out);
   input clk,rst,en;
   output [3:0] out;
         wire [3:0] out;
         reg [3:0] count;
         always@(posedge clk)
         if(rst)
         count<=0;
         else if(en)
         count<=count+1;
         assign
out={count[3],count[3]^count[2],count[2]^count[1],count[1]^count[0]};


endmodule
```

